

Vue3を使ったTODOリストを作成しよう

目的

vueの簡単なアプリを理解するため、TODOリストを作成する。

以下のリポジトリからクローンを行い、課題を達成しよう。

<https://github.com/nichide-beni/node-todo-app.git> また課題については簡単なものに設定しているので、コメントなどを確認しつつアプリの全体像を理解することが最大目的である。

課題

削除機能を使った時、別なページに遷移してから一覧ページに戻ると削除したはずのデータが残っている。正しく削除が反映される方法を考えてみよう。

課題解決のヒント

vueのアプリで解決しよう。

vueの公式チュートリアルの問題で同じような解答例がある。

ディレクトリ構造

```
├── README.md # READMEにしたがって、npm installなどを実行しよう(プロジェクト生成者のみの項目はスキップ)
├── client # Vueのアプリ(メイン)
│   ├── README.md # デフォルトのため見なくていい
│   ├── index.html # Vueの展開場所、SPA形式でidを設定するくらいなので一読でOK
│   ├── package-lock.json
│   ├── package.json
│   ├── public # 公開サーバーのアセットデータなど、今回はデフォルトのファビコンのみ。
│   │   └── vite.svg
│   ├── src
│   │   ├── App.vue # Vueアプリの共通設定などを記述する
│   │   ├── assets #内部で参照する場合のアセットデータなど
│   │   │   └── vue.svg
│   │   ├── components # 本来はこの内部にパーツを格納。今回で言うと、うまくやれば
│   │   │   createとeditくらいはまとめられる。
│   │   ├── controller
│   │   │   └── todoApi.js # expressサーバーへの接続
│   │   ├── main.js # Vueの立ち上げ
│   │   ├── router
│   │   │   └── index.js # vue-routerを使ったルーティングの実体。SPAの遷移や名前の
│   │   │   設定を定義する。
│   │   ├── style.css
│   │   └── views # 実際の画面。HTMLなどのレンダリングに相当する。
│   │       ├── TodoCreate.vue # TODOの作成画面
│   │       ├── TodoEdit.vue # 作成済みTODOの編集画面
│   │       └── TodoList.vue # TODOリストの表示画面(削除、編集などの遷移も担当)
│   └── vite.config.js # viteのコンパイル設定、import用の@エイリアスを追記
└── server # expressのアプリ(編集なし、立ち上げのみ)
```

```
├── database.js # SQLiteサーバーへの接続プログラム
├── index.js # ルーティングファイル、APIのエンドポイント
├── local_data
│   └── todo.db # SQLiteの実体(編集不可)
├── package-lock.json
└── package.json # スクリプトはこちらを参照
```

アプリの説明

以下の機能を有する。

- TODOアプリのホーム画面(起動時のみ)
- TODOリストの表示
- 新しいTODOを作成
- 作成済みのTODOを編集
- 作成済みのTODOを完了
- 作成済みのTODOを削除(未完成)

全体像としては次のとおり確認してみよう(client/src以下)。

1. App.vueで共通設定を確認
2. routes/index.jsでアプリ全体の構造(サイトマップに近いもの)を把握
3. todoApi.jsでexpressとの通信内容を把握
4. views/の各ファイルを確認し、それぞれの画面の役割を確認
5. アプリを動かしながら、どういうコードで動いているか
6. 最後に課題内容を適用して完成

expressサーバーの説明

メイン処理としては「index.js」で完結するシンプルなAPIサーバー。

エラー処理などはかなり省いているため、本番環境には適さない。

Vueアプリと1:1で動作する形式で、表示、作成、更新、削除の機能とデータベース機能を提供する。

app.get, app.postなどの('/todos/:id')などはURIに該当し、固定文字がURIとして、「:id」など「:」つきのものは変数(ID値)を入力する。

プログラム内の「body」から取得する値については、POSTで送信された値である。

※edit機能を参照。URIからはidを取得し、その他のパラメータはPOSTされたbodyから取得している。